

Catalogizer

Продвинутое управление медиаколлекциями с поддержкой нескольких протоколов — обнаружение, каталогизация и обогащение всего, чем вы владеете.

Краткое описание

Catalogizer — это система управления медиаколлекциями с возможностью самостоятельного хостинга, которая автоматически обнаруживает, классифицирует и упорядочивает медиафайлы на SMB, FTP, NFS, WebDAV и локальных файловых системах. Она обеспечивает мониторинг в реальном времени, шифрованное хранение данных, обогащение внешними метаданными и современный интерфейс React, работающий на базе высокопроизводительного бэкенда Go API.

Короткое описание

Многофункциональный менеджер медиабibliothек промышленного уровня. Go/Gin REST API распознаёт более 50 типов медиафайлов из источников SMB/FTP/NFS/WebDAV/ локальных хранилищ, обогащает данные из TMDB, IMDb, MusicBrainz, Steam и других сервисов, а также предоставляет веб-приложение React в реальном времени через зашифрованную базу данных SQLCipher.

Подробное описание

Большинство медиаменеджеров требуют от вас сначала капитулировать: свести всё в одно место, в одном формате, одного типа — и только тогда они согласятся помочь. Catalogizer исходит из противоположной предпосылки: ваша коллекция уже существует там, где существует, разбросанная по сетевым хранилищам и протоколам, которые никогда не договорятся между собой, — и именно там система её и находит. Она поддерживает протоколы, которые уже используются для хранения данных: SMB/CIFS, FTP/FTPS,

NFS, WebDAV и локальные файловые системы — за единой унифицированной абстракцией клиента. Благодаря этому сетевая папка Windows, FTP-архив и монтированный WebDAV-ресурс выглядят одинаково для вышестоящих уровней и могут свободно комбинироваться, заменяться или выводиться из эксплуатации без изменений в коде приложения. Бэкенд Go (Gin REST API) непрерывно отслеживает эти источники, обнаруживает и классифицирует более 50 типов медиафайлов (фильмы, сериалы, музыка, игры, ПО, документальные проекты) по мере их появления, а затем обогащает каждый элемент данными из внешних источников — TMDb, IMDb, TVDB, MusicBrainz, Spotify, Steam и других. В результате из простого имени файла формируется полноценная запись в каталоге с обложками, информацией о создателях и метаданными. Результаты передаются во фронтенд TypeScript React через WebSockets, поэтому библиотека обновляется в реальном времени по мере обработки данных, а все метаданные сохраняются в зашифрованной базе SQLCipher, доступ к которой контролируется системой аутентификации на основе JWT с ролевым разграничением прав.

В то время как большинство каталогизаторов теряют работоспособность при малейшем сбое в доступе к ресурсу, Catalogizer спроектирован так, чтобы оставаться полезным даже в условиях неполадок. Временный сбой SMB-соединения компенсируется экспоненциальным откатом при повторных попытках подключения, механизмом предохранителя, блокирующим атаки на неработающий хост, непрерывным мониторингом состояния и кэшем метаданных в офлайн-режиме, который продолжает обслуживать пользовательские запросы на основе последнего актуального состояния. Разница между «всё приложение не работает, потому что перезагрузился один NAS» и «один источник недоступен, но всё остальное функционирует» — принципиальна. Помимо каталогизации, система служит инструментом для управления коллекцией: аналитика роста и отслеживания версий/качества, генерация профессиональных отчётов PDF, сервис конвертации PDF в изображения/текст/HTML, экспорт/импорт избранного (JSON/CSV), а также синхронизация с облачными хранилищами S3, Google Cloud Storage или локальными папками. При этом Catalogizer — не монолит, случайно разросшийся до больших размеров, а продуманная сборка из 21 переиспользуемого подмодуля `digital.vasic.*` Go и клиентских пакетов TypeScript, каждый из которых проходит независимое тестирование и версионирование. Те же проверенные в боях компоненты аутентификации, работы с файловой системой, потоковой передачи данных и мониторинга, которые обеспечивают работу Catalogizer, используются и в других продуктах семейства. Контроль качества не ограничивается декларациями: фреймворк

Challenges и HelixQA проверяют каждую заявленную функцию на соответствие реальным возможностям, подтверждая их документально.

Зачем мы его создали

Существующие медиаменеджеры рассчитаны на один тип хранилища и один формат контента. Настоящие коллекции разбросаны по множеству NAS-накопителей и протоколов, теряют работоспособность при сбоях в доступе и объединяют фильмы, музыку, игры и программы. Catalogizer разработан, чтобы одинаково работать со всеми протоколами, выдерживать нестабильные сетевые хранилища и предоставлять единый авторитетный, обогащённый и зашифрованный каталог всего контента.

Почему это меняет правила игры

Он объединяет в одном самодостаточном зашифрованном пакете то, для чего обычно требуется целый стек отдельных инструментов: универсальный инжиниринг, одинаково работающий с любым типом хранилищ, устойчивость, позволяющую каталогу оставаться доступным даже при сбоях хранилищ, и мощную систему обогащения данных от множества провайдеров, превращающую сырые файлы в структурированную библиотеку с атрибутами. Преимущество модульной архитектуры — в её кумулятивном эффекте: исправление в клиенте файловой системы или новый плагин провайдера внедряются один раз и улучшают работу всех потребителей. Таким образом, Catalogizer становится всё лучше по мере развития окружающей экосистемы. Короче говоря, это разница между медиаиндексом и *медиасистемой* — системой, которая принадлежит вам, выдерживает нестабильную инфраструктуру и основана на проверенных, а не обещанных решениях.

Что нового

- Единый клиент файловой системы с поддержкой нескольких протоколов (SMB/FTP/NFS/WebDAV/локальные хранилища) за одним интерфейсом.
- Офлайн-кэш и автоматический выключатель, благодаря которым каталог остаётся работоспособным даже при сбоях хранилищ.
- Полное разделение на 21 переиспользуемый подмодуль `digital.vasic.*` и клиентские модули на TypeScript.

- Каталог с шифрованием на уровне хранения (SQLCipher) и синхронизацией в реальном времени (WebSocket) с интерфейсом.
- Контроль качества на основе доказательств через фреймворк Challenges и интеграцию с HelixQA.

Проблемы и решения

- **Нестабильные сетевые хранилища:** решается за счёт экспоненциального отката, автоматического выключателя, проверок работоспособности и офлайн-кэша с политикой вытеснения, который предоставляет кэшированные метаданные при недоступности источников.
- **Разнородность протоколов:** решается абстрагированием всех протоколов за общим клиентом `digital.vasic.filesystem`, благодаря чему вышестоящие уровни не зависят от конкретного протокола.
- **Безопасность данных:** обеспечивается шифрованием на уровне хранения (SQLCipher) в сочетании с аутентификацией JWT/RBAC и промежуточным ПО для санации запросов.
- **Масштабируемость и поддержка:** достигается за счёт выделения всей универсальной логики в независимо тестируемые подмодули вместо монолитной архитектуры.

Технологический стек (зачем и как)

- **Go + Gin** — высокопроизводительное ядро REST API (`catalog-api`); выбрано за счёт высокой конкурентности и пропускной способности при непрерывном мониторинге.
- **TypeScript + React + Tailwind (Vite)** — адаптивный веб-интерфейс `catalog-web` с обновлениями в реальном времени.
- **WebSockets** — синхронизация данных в реальном времени между бэкендом и интерфейсом.
- **SQLCipher (зашифрованное хранилище SQLite)** — зашифрованное хранилище метаданных с поддержкой SQLite/PostgreSQL через `digital.vasic.database`.
- **Клиенты SMB/FTP/NFS/WebDAV** — мультипротокольный инжиниринг через `digital.vasic.filesystem`.
- **Внешние API метаданных (TMDB, IMDB, TVDB, MusicBrainz, Spotify, Steam)** — плагины провайдеров для обогащения данных.
- **Prometheus + OpenTelemetry** — метрики и трассировка через `digital.vasic.observability`.
- **Docker / контейнер сборки** — воспроизводимые сборки (Tauri/Rust через `catalogizer-builder`).
- **Redis** — кэширование и ограничение частоты запросов через `digital.vasic.cache / ratelimiter`.

- **S3 / Облачное хранилище Google** — облачная синхронизация и хранение контрольных точек.